



OS-Project3

주제

Memory-Mapped files

작성자

전예원 / 컴퓨터소프트웨어학부 / 2023096608 / 3학년 / 12913

목차

1. Design
2. Implementation
3. Results
4. Troubleshooting

1. Design

과제명세서에 나와있는 step별로 디자인을 생각해보았다.

sys_mmap

args를 검증하고, VMA slot에서 빈 공간을 찾은 뒤 TRAPFRAME 아래, $p \rightarrow \text{size}$ 보다 위에서 사용하지 않는 VA를 선택해서 VMA를 채운 뒤 file ref 증가를 위해 fileup(f)를 호출하여 선택한 주소를 반환하는 구조이다.

- 먼저, 해당 함수에서 len, prot, flags, fd, offset에 올바른 값이 들어왔는지 검사한다.
- 그리고 빈 VMA를 찾고 mmap 영역 주소를 고른다.
- 주소가 올바르게 선택되면 vma의 flag(used, addr, length 등)에 인자로 들어온 값과 알맞은 값들을 setting 해준다.
- 이후 vma의 addr을 return해주면 된다.

mmapfault

mmap 영역에 접근하여 page fault가 발생하였을 때 호출하는 함수이다.

- va를 포함하는 VMA를 찾는다.
- physical page를 할당한 뒤 0으로 초기화하고, file에서 해당 위치의 data를 읽어온다.
- prot를 PTE permission으로 변환한다.
- fault가 난 주소에 physical page를 mapping하고 매핑된 physical address를 반환하게 하면 된다.

파일이 page보다 작거나 EOF와 가까우면 4KB를 다 못읽을 수 있는데, 이때 남은 부분은 0으로 남겨두게 처리하면 된다.

munmap과 write-back

munmap으로는 VMA를 지우는 것 뿐만 아니라 page table에 올라온 page가 있으면 이것도 함께 제거해야한다.

- 먼저, MAP_SHARED이고 실제 page가 존재하면 파일에 write-back한다.
- physical page를 free하고 PTE를 clear해준다.
- VMA의 length, addr, offset을 조정해준다.
- 전체 VMA가 사라졌으면, file close후 VMA slot을 해제해주면 된다.

한번도 접근하지 않은 mmap page는 page fault가 발생하지 않았으므로 실제 PTE가 없을 수도 있다. 이 경우에는 skip하도록 해주면 된다.

middle-hole unmap은 거부하도록 해주면 된다. 즉, VMA 중간만 뚫으려하면 -1을 반환하면 된다.

kfork와 kexit 수정

kfork의 경우, 부모가 mmap한 영역을 자식 process도 볼 수 있어야하므로, 부모의 VMA array의 copy가 필요하다. physical page는 자식과 공유하지 않으므로, 부모가 읽어들인 page가 있어도 자식은 VMA 정보만 가지고 있다가 해당 주소 접근시 page fault 발생하고 파일에서 읽어오도록 해주어야한다.

VMA는 file를 가리키고 있으므로, 자식 process의 VMA도 파일 refcount가 유지되도록 filedup 처리가 필요해 보였다.

kexit의 경우, process 종료 시 mmap 영역이 남아있으면 정리해주어야한다.

- MAP_SHARED로 매핑한 영역에서 수정된 내용은 process 종료 전 파일에 반영되어야한다.
- 각 VMA에 대해 munmap과 비슷한 write-back + unmap 처리를 해주면 된다.

대신, pagetable이 free되기 전에 해당 처리를 해야하는 건 필수이다.

2. Implementation

sys_mmap 구현

kernel/sysfile.c

```
uint64
sys_mmap(void)
{
    int len, prot, flags, fd, offset;
    struct file* f;
    struct vma* target = 0;
    struct proc* p = myproc();

    argint(0, &len);
    argint(1, &prot);
    argint(2, &flags);
    if(argfd(3, &fd, &f) < 0) return -1;
    argint(4, &offset);
```

```

//인자 검증
if(len <= 0 || len % PGSIZE != 0) return -1;
if(prot == PROT_NONE || (prot & ~(PROT_READ | PROT_WRITE)) != 0) return
-1;
if(flags != MAP_SHARED && flags != MAP_PRIVATE) return -1;
if(offset != 0) return -1;

if(f->readable == 0) return -1; //읽기 권한 가진 것만
if((prot & PROT_WRITE) && flags == MAP_SHARED && f->writable == 0) retur
n -1;
//MAP_SHARED는 쓰기 권한도 있어야함

//빈 VMA slot 찾기
for(int i=0; i<NVMA; i++){
    if(p->vmass[i].used == 0){
        target = &p->vmass[i];
        break;
    }
}
if(target == 0) return -1; //빈 slot 없음!

uint64 top = TRAPFRAME;
for(int i=0; i<NVMA; i++){
    if(p->vmass[i].used && p->vmass[i].addr < top) top = p->vmass[i].addr;
}

if((uint64)len > top) return -1; //유효한 addr인지 판단
if(top-(uint64)len < p->sz) return -1;

uint64 addr = top - len;

target->used = 1;
target->addr = addr;
target->length = len;
target->prot = prot;
target->flags = flags;
target->offset = offset;
target->f = filedup(f);

return addr;
}

```

- 먼저 syscall을 통해 받는 인자인 len, prot, flags, fd, offset 변수로 설정하여 해당 인자를 argint와 argfd로 받아온다.
 - fd 숫자를 받아서 ofile[fd]에서 실제 file을 찾아오도록 argfd를 통해 처리해주었다.
- 그리고 명세서에 주어진 조건대로 인자를 검증해주었다.

- 추가적으로, 파일 권한에 대해서 검증도 필요하다고 생각하여 read할 수 있으면서, MAP_SHARED인 경우 memory에 올라온 부분이 수정되면, 이를 file에도 반영해야하므로 write 권한이 있는지 확인해주었다.
- 그리고 for문을 돌려 빈 VMA slot을 찾아주었다. 빈 slot을 못찾았으면 -1을 return해주면 된다.
 - 프로세스마다 struct vma vmas[NVMA]이 있고 최대 16개의 mapping을 관리함을 인지하면 된다.
- 이후 mmap하는 주소를 고르면 되는데, TRAPFRAME 아래에 위치하면서, 아래 방향으로 grow한다. 이미 mapping된 VMA가 있으면 가장 낮은 시작 주소를 top으로 잡고, 새 mapping은 그 아래에 붙이면 된다.
- 이후, 해당 주소가 유효한지 확인해준다.
- 위의 모든 것을 확인하면, VMA를 채워주면 된다.
 - mmap() 직후 close(fd)를 해도, mapping은 살아있어야하므로 VMA가 파일 reference를 하나 더 들고 있어야한다. 따라서 f에 fildup(f)을 호출해서 해당 return 값을 넣어주면 된다.
- 최종적으로, 시작 주소인 addr을 반환해주면 된다.

mmapfault 구현

kernel/sysfile.c

```
int
mmapfault(uint64 va)
{
    struct proc *p = myproc();
    struct vma *v = 0;
    uint64 startva = PGROUNDOWN(va); //page 시작주소로 매핑해야하므로

    //fault address가 난 VMA 찾기
    for(int i=0; i<NVMA; i++){
        if(p->vmas[i].used && p->vmas[i].addr <= startva && p->vmas[i].addr +
p->vmas[i].length > startva){
            v = &p->vmas[i];
            break;
        }
    }
    if(v == 0) return -1; //못찾으면

    if(ismapped(p->pagetable, startva)) return -1; //이미 매핑되었는지 확인 필
수!!

    //physical page 할당 후 초기화
    char *mem = kalloc();
    if(mem == 0) return -1;
    memset(mem, 0 ,PGSIZE);

    if(v->f->type != FD_INODE){
        kfree(mem);
        return -1;
    }
}
```

```

uint offset = v->offset + (startva - v->addr);

ilock(v->f->ip);
int i = readi(v->f->ip, 0, (uint64)mem, offset, PGSIZE);
iunlock(v->f->ip);

if(i<0){
    kfree(mem);
    return -1;
}

//PTE permission
int perm = PTE_U;
if(v->prot & PROT_READ) perm |= PTE_R;
if(v->prot & PROT_WRITE) perm |= PTE_W;

//page table에 매핑하기
if(mappages(p->pagetable, startva, PGSIZE, (uint64)mem, perm) < 0){
    kfree(mem);
    return -1;
}
return 0;
}

```

- 먼저 parameter로 들어온 va를 page단위로 보기 위해 PGROUNDDOWN 함수를 이용하여 startva를 뽑아 주었다. 실제 매핑은 page 시작주소에서 해야하기 때문이다.
- 그리고, 해당 주소가 어떤 VMA에 속하는지 for문을 통해 찾아준다. sys_mmap에서 mapping 의도를 기록했으므로, 해당 함수에서는 이 fault가 mmap 영역에서 난 fault인지 확인하고, 못찾으면 실패이다.
- 그리고 이미 매핑된 page인지 ismapped를 통해 검사해주었다.
- mem이라는 변수에 physical page를 할당하고, 0으로 먼저 초기화해준다.
 - 과제 명세에서 한 페이지보다 적게 읽을 수도 있으니, 남은 부분은 zero-fill하라고 했다. 이후 파일을 읽고, 남은 부분은 자동으로 0이 채워져있으니 처음부터 0으로 초기화해주는 방식이 좋다고 생각하였다.
- 이후 파일 offset을 계산해주었다. fault가 첫번째 페이지가 아닌 다른 페이지에서 났다면, 파일에서도 해당 위치부터 읽어야한다.
- 이후 fileread 함수를 참고하여 inode 파일 읽기를 readi로 구현해주었다. kernel memory로 읽어야하므로 2번째 인자를 0으로 두면 된다.
- PROT_를 PTE_로 바꿔서 매핑해주었다. 명세서에서 직접 변환해야한다고 되어있어 직접 변환 로직을 넣어서 PTE permission을 해주었다.
 - PTE_U는 user process가 접근하는 것이므로 항상 필요하다.
- 마지막으로, 실제 page table에 연결하면 된다.

sys_munmap 구현

```

uint64
sys_munmap(void)
{
    uint64 addr; int len;
    struct proc* p = myproc();
    struct vma* v = 0;

    argaddr(0, &addr);
    argint(1, &len);

    //값 유효한지 검사
    if(addr % PGSIZE != 0) return -1;
    if(len <= 0 || len % PGSIZE != 0) return -1;

    uint64 end = addr + len;
    //VMA 찾기
    for(int i=0; i<NVMA; i++){
        if(p->vmass[i].used && p->vmass[i].addr <= addr && p->vmass[i].addr + p->
vmass[i].length >= end){
            v = &p->vmass[i];
            break;
        }
    }
    if(v==0) return -1;

    uint64 curstart = v->addr;
    uint64 curend = v->addr + v->length;

    //중간 hole은 금지!
    if(addr!=curstart && end != curend) return -1;

    //page 처리
    for(uint64 i = addr; i<end; i += PGSIZE){
        pte_t *pte = walk(p->pagetable, i, 0);

        if(pte == 0 || (*pte & PTE_V) == 0) continue;

        uint64 pa = PTE2PA(*pte);

        if(v->flags == MAP_SHARED){
            uint offset = v->offset + (i - v->addr);

            begin_op();
            ilock(v->f->ip);
            int n = writei(v->f->ip, 0, pa, offset, PGSIZE);
            iunlock(v->f->ip);
        }
    }
}

```

```

        end_op();

        if(n<0) return -1;
    }
    uvmunmap(p->pagetable, i, 1, 1);
}

//정보 수정
if(addr == curstart && end == curend){
    fileclose(v->f);
    v->used = 0;
    v->f = 0;
    v->addr = 0;
    v->length = 0;
    v->offset = 0;
    v->prot = 0;
    v->flags = 0;
}else if(addr == curstart){
    //앞 부분을 unmap
    v->addr = end;
    v->offset += len;
    v->length -= len;
}else{
    //뒷 부분 unmap
    v->length -= len;
}
return 0;
}

```

- user API가 munmap(void *addr, int len); 이므로 addr은 argaddr로, len은 argint로 인자 받아주었다.
- 이후 인자를 검사해주는데, len은 양수이고 PGSIZE 배수라고 했으므로 해당 부분을 검사해주었고, addr도 page 단위의 mapping을 제거하는 것이므로 PGSIZE 배수이어야 함을 검사해주면 된다.
- VMA 찾기는 요청 범위 전체가 VMA에 들어가는지를 봐야하므로 해당 부분을 검사하며 for문으로 찾아주면 된다.
- 이후 과제 명세에서 중간 hole 금지이므로 자르기가 허용되는 범위는 다음과 같다.
 - 앞에서 자르기: addr == curstart
 - 뒤에서 자르기: end == curend
 - 전체를 제거
- page 처리에서는 walk를 통해 PTE를 보면 되는데, 우리는 lazy map을 쓰므로 접근하지 않은 page는 physical page가 없다. 따라서 그냥 넘어가도 된다. 이를 continue로 처리해주었다.
- 그리고 unmap시, MAP_SHARED이면 file에 써야하므로, offset과 physical address를 계산하고 얻어와서 write를 통해 넘겨서 파일에 써주면 된다.
- 이후, 실제 page table에서 제거하고 physical page를 free해주기 위해 uvmunmap을 호출해주었다.
 - 마지막 do_free에 1을 넣어 physical page도 제거해주면 된다.

- 이후 VMA 정보를 update해주어야한다. 자르기가 허용되는 범위 3가지를 나누어 각각 조건에 맞게 setting 해주었다.
 - 앞에서 잘랐으면, $v \rightarrow \text{addr}$ 은 end 주소를 넣어주면 되고, length는 len만큼 줄어든다. 그리고 offset은 len만큼 더해주면 된다.
 - 뒤에서 잘랐으면 $v \rightarrow \text{addr}$ 과 $v \rightarrow \text{offset}$ 은 그대로이지만, $v \rightarrow \text{length}$ 만 len만큼 줄어든다.
 - 전체 unmap이면 VMA가 사라지는 것이니 `fileclose()`를 호출하여 refcount를 내려주어야한다.

fork와 exit 수정

kernel/proc.c

kfork

```
int
kfork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    ...

    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);

    //VMA 배열 복사 필요
    for(i = 0; i<NVMA; i++){
        np->vmass[i] = p->vmass[i];
        if(np->vmass[i].used) filedup(np->vmass[i].f);
    }

    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;

    ...

    return pid;
}
```

kfork의 특징은, 부모의 VMA array를 복사하고, physical pages를 share하지 않는다는 뜻이다.

- 그래서 NVMA만큼 순회하며 부모의 vmass 배열을 copy해주고, refcount도 올려주어야하므로 filedup을 used하고 있는 부분에 대해서 호출해주면 된다.
- 즉, child는 해당 VMA 정보만 가지고 있다가 page fault가 발생하면 그때 file을 읽어오면 된다.

exit

```

#include "fs.h"
#include "sleeplock.h"
#include "file.h"
#include "fcntl.h"

void
kexit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    for(int i = 0; i<NVMA; i++){
        struct vma* v = &p->vmass[i];

        if(v->used == 0) continue;

        for(uint64 a = v->addr; a<v->addr+v->length; a += PGSIZE){
            pte_t *pte = walk(p->pagetable, a, 0);

            if(pte == 0 || (*pte & PTE_V) == 0) continue;

            uint64 pa = PTE2PA(*pte);

            if(v->flags == MAP_SHARED){
                uint64 offset = v->offset + (a - v->addr);

                begin_op();
                ilock(v->f->ip);
                int n = writei(v->f->ip, 0, pa, offset, PGSIZE);
                iunlock(v->f->ip);
                end_op();

                if(n<0) panic("kexit writeback");
            }

            uvmunmap(p->pagetable, a, 1, 1);
        }

        fileclose(v->f);
        v->used = 0;
        v->f = 0;
        v->addr = 0;
        v->length = 0;
        v->offset = 0;
        v->prot = 0;
    }
}

```

```

    v->flags = 0;
}

// Close all open files.
for(int fd = 0; fd < NOFILE; fd++){
    if(p->ofile[fd]){
        struct file *f = p->ofile[fd];
        fclose(f);
        p->ofile[fd] = 0;
    }
}

...
}

```

전체 VMA를 항상 정리해야하므로, `sys_munmap()`과 유사한 코드를 추가해주면 되었다. 이를 처리하기 위해서는 4개의 header를 먼저 추가해서 선언해주어야한다.

- 현재 process가 가진 VMA 전체를 보는데 `used`하지 않으면 건너뛰어도 된다.
- 그리고 virtual address 범위를 `v->addr`와 `v->addr+v->length`로 잡아주면 된다. 그리고 `PGSIZE` 단위로 순회하며 해당 page가 실제로 mapping되어있는지를 확인하여, `pte == 0`이면 page table entry 자체가 없고, valid mapping이 아닌 경우도 `continue`해주면 된다.
- 이후 PTE에서 physical address를 꺼내어 `MAP_SHARED`인 경우 변경 내용이 파일에 반영되게 해주면 된다.
 - write 실패 시 panic으로 처리하였다.
- 그리고 page table에서 해당 virtual page mapping을 제거하고, physical page도 free하도록 `uvmunmap`을 호출해주었다.
- 모든 page를 처리한 뒤, `fclose`를 통해 file refcount를 내려주고 VMA slot을 초기화해주었다.

3. Results

step1 구현 후 결과

```

test basic mmap
usertrap(): unexpected scause 0xd pid=3
sepc=0x10a stval=0x3ffffc000

```

이는 `mmap()` 자체는 성공하였으나, 첫 메모리 접근에서 `mmapfault()`가 page fault를 처리하지 못해 kill되어 기대하는 결과와 일치하였다.

step2,3 구현 후 결과

```
test basic mmap: OK
test mmap private: OK
test mmap read-only: OK
test mmap read/write: OK
test mmap dirty: OK
test not-mapped unmap: OK
test lazy access: OK
test mmap two files: OK
test fork
usertrap(): unexpected scause 0xd pid=4
sepc=0x10a stval=0x3ffffc000
mmaptest: test fork failed: child exit, pid=3
```

이는 single-process map, munmap, write-back 테스트는 통과했고, test fork에서 실패하고 있음을 알려준다. 과제 명세서에서 test fork는 여전히 실패한다고 했으므로 기대하는 결과와 일치하였다.

step4까지 구현 후 최종 결과

mmaptest와 usertests -q까지 모두 통과하는 것을 확인하였다.

```
j@DESKTOP-NMJNJ01:~/project3-dhkdxb$ make qemu CPUS=1
```

```
test basic mmap: OK
test mmap private
test mmap private: OK
test mmap read-only
test mmap read-only: OK
test mmap read/write
test mmap read/write: OK
test mmap dirty
test mmap dirty: OK
test not-mapped unmap
test not-mapped unmap: OK
test lazy access
test lazy access: OK
test mmap two files
test mmap two files: OK
test fork
test fork: OK
test munmap prevents access
usertrap(): unexpected scause 0xd pid=5
          sepc=0xa6e stval=0x3fffffd000
usertrap(): unexpected scause 0xd pid=6
          sepc=0xae8 stval=0x3fffffc000
test munmap prevents access: OK
test writes to read-only mapped memory
usertrap(): unexpected scause 0xf pid=7
          sepc=0xc3e stval=0x3fffffc000
test writes to read-only mapped memory: OK
mmaptest: all tests succeeded
```

```
PASS
test lazy_copy: PASS
test lazy_sbrk: PASS
PASS ALL TESTS
```

4. Troubleshooting

mmapfault 구현 시 이미 매핑된 페이지 확인 누락

해당 함수에서 startva가 어떤 VMA에 속하는지 찾아주고 바로 물리 페이지 할당해서 작업을 해주었다. 하지만, 여기서 놓친 부분이 있었다. 이미 매핑된 page인지 확인을 먼저 해주어야했다. 만약, PROT_READ로 mmap한 주소에 write를 하려고 하면 처음에는 read-only page가 이미 매핑되어 있을 수 있다. 그럼 write가 안되므로, 권한 위반에 따른 fault가 날 것이다. 이는 새로 페이지를 만드는 상황이 아니므로 해당 조건을 filtering 해주어야했다. 처음에는 이를 하지 않아 mmaptest에서 mappages:remap panic이 났었다. 이를 해결하기 위해 오랜 시간 고민하

였고, 결과적으로 권한에 맞지 않는 fault로 2번 mappages 호출이 문제임을 알게 되었다. 조금 더 세심한 조건 check가 필요함을 깨닫게 되었다.